# Additional Information on Docker

# 1. PostgreSQL can be created automatically

Normally, you might open **pgAdmin** and manually create a database:

```
pgAdmin
   ↓
PostgreSQL
   ↓
Create Database
   ↓
mydb
```

But when using a PostgreSQL Docker container, you can configure the database during container startup.

For example, Docker Compose might contain something like:

YAML

```yaml
services:
  postgres:
    image: postgres
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
```

Here:

```
POSTGRES_USER      → creates/configures the PostgreSQL user
POSTGRES_PASSWORD  → password for that user
POSTGRES_DB        → database to create
```

The standard PostgreSQL Docker image creates the database specified by:

```yaml
POSTGRES_DB
```

If `POSTGRES_DB` isn't specified, the image uses the value of `POSTGRES_USER` as the default database name during initialization.

So the safer interview statement is:

> **"The official PostgreSQL Docker image can automatically initialize a database during first-time container initialization using `POSTGRES_DB`. If `POSTGRES_DB` is omitted, the default database name is typically the value of `POSTGRES_USER`."**

# 2. Overall Flow

The overall flow is:

```
Docker Compose
      |
      v
PostgreSQL Container
      |
      +---- PostgreSQL User
      |
      +---- PostgreSQL Database
      |
      +---- PostgreSQL Server
                |
                v
          Spring Boot App
```

Your Spring Boot application connects to PostgreSQL using the JDBC URL.

# 3. Why Do We Need PostgreSQL?

### PostgreSQL

PostgreSQL is the **actual database server/database system**.

### pgAdmin

pgAdmin is a **GUI administration tool** for PostgreSQL.

Think:

```
PostgreSQL
    ↓
Actual database engine

pgAdmin
    ↓
GUI used to manage/inspect PostgreSQL
```

You don't need pgAdmin for PostgreSQL itself to work.

You can interact with PostgreSQL using:

- `psql`
- JDBC
- Spring Data JPA/Hibernate
- Other database clients
- pgAdmin

# 4. Why use PgAdmin

Instead of using command line:

```bash
psql
```

you can use pgAdmin to visually see:

```
Servers
   ↓
PostgreSQL
   ↓
Databases
   ↓
markx
   ↓
Schemas
   ↓
public
   ↓
Tables
```

You can inspect:

- Databases
- Tables
- Columns
- Rows
- SQL queries
- Constraints
- Indexes
- Relationships

So pgAdmin is basically a **GUI management tool**.

# 5. Important Distinction: PgAdmin Vs. PostgreSQL

**Interview question:**

**"Is pgAdmin the database?"**

**Answer:**

> **No. PostgreSQL is the database management system, while pgAdmin is a graphical administration and management tool used to interact with PostgreSQL.**

Very important.

# 6. What happens when SpringBoot creates Table?

Java

```java
@Entity
public class User {

    @Id
    private Long id;

    private String name;
}
```

Java

```java
@Entity
public class Product {

    @Id
    private Long id;

    private String name;
}
```

Hibernate/JPA can create tables such as:

```
user
product
```

inside your PostgreSQL database depending on your configuration.

For example:

properties

```
spring.jpa.hibernate.ddl-auto=update
```

Then:

```
Spring Boot
     |
     v
Hibernate/JPA
     |
     v
PostgreSQL
     |
     +---- user table
     |
     +---- product table
```

# 7. Why did the data appear in PGADMIN?

The instructor creates:

```
1 User
1 Product
```

through the application.

Then he opens pgAdmin and sees those records.

Why?

Because:

```
Application
    ↓
Spring Data JPA / Hibernate
    ↓
JDBC
    ↓
PostgreSQL
    ↓
Database
    ↓
Tables
```

The application is writing directly into the PostgreSQL database.

pgAdmin is simply showing what's already stored there.

# 8. Docker Troubleshooting

For example:

```
Container won't start
Docker behaves strangely
Images/containers have conflicts
Old data remains
Docker installation has issues
```

Docker Desktop provides troubleshooting/reset options.

# 9. On Windows

```
Docker Desktop
      ↓
Troubleshoot
```

Depending on Docker Desktop version, you may see options related to:

- Restarting Docker
- Cleaning data
- Purging data
- Resetting Docker
- Factory reset

The exact menu/options can vary by Docker Desktop version.

# 10. What does clean/purge data means?

This is where you need to be careful.

Cleaning Docker data can remove Docker-managed resources/data.

Depending on the option used, this can affect:

- Containers
- Images
- Volumes
- Build cache
- Other Docker data

So **don't casually click purge/reset** on a system containing important Docker data.

**Think of it as:**

```
Docker accumulated old/unused/conflicting data
                  ↓
          Cleanup may help
                  ↓
        Docker starts fresh
```

# 11. Factory Reset

A factory reset is much more aggressive.

Conceptually:

```
Docker Desktop
      ↓
Factory Reset
      ↓
Docker returns toward initial/default state
```

This can remove Docker data and configuration.

### Interview point

You don't need to memorize the exact Docker Desktop button.

Understand the principle:

> **When Docker Desktop has corrupted/conflicting local state, restarting, cleaning unused data, or resetting Docker can sometimes resolve the issue.**

# 12. Dangling Data

Docker can accumulate unused resources such as:

- Dangling images
- Build cache
- Stopped containers
- Unused networks
- Unused volumes

These can sometimes contribute to confusion or consume disk space.

Docker provides cleanup commands such as:

```bash
docker system prune
```

But be careful.

It can remove unused Docker resources.

# 13. PostgreSQL Port conflict

```
Local PostgreSQL
       |
       ↓
localhost:5432  ← occupied
       ↑
Docker PostgreSQL
       |
       ↓
Cannot bind port
```

Solution:

- Stop local PostgreSQL, **or**
- Map Docker PostgreSQL to another host port.

# 14. Docker itself has an issue

```
Docker Desktop
      ↓
Container doesn't start properly
```

Possible troubleshooting:

- Restart Docker
- Check container logs
- Check Docker status
- Clean unused Docker data
- Reset Docker as a last resort

# 15. Troubleshooting Checklist

### Step 1 — Is Docker running?

```
Docker Desktop → Running?
```

### Step 2 — Is the PostgreSQL container running?

Run:

```bash
docker ps
```

Look for your PostgreSQL container.

### Step 3 — Check container logs

```bash
docker logs <container-name>
```

Look for:

```
database system is ready to accept connections
```

This is a good sign.

### Step 4 — Check port mapping

Run:

```bash
docker ps
```

You might see:

```
0.0.0.0:5432->5432/tcp
```

This means:

```
Host 5432 → Container 5432
```

### Step 5 — Check whether local PostgreSQL is using the port

On Windows:

```
Services
   ↓
PostgreSQL
```

Stop it if you don't need it.

Or change the Docker host port:

```yaml
ports:
  - "5433:5432"
```

## Step 6 — Check Spring Boot configuration

For example:

properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=myuser
spring.datasource.password=mypassword
```

Make sure these match the Docker PostgreSQL setup.

# 16. Revision

```
POSTGRESQL + DOCKER
_____________________________


Docker PostgreSQL
    ↓
Can initialize:
    POSTGRES_USER
    POSTGRES_PASSWORD
    POSTGRES_DB

Spring Boot
    ↓
JDBC
    ↓
PostgreSQL
    ↓
Database
    ↓
Tables + Data

pgAdmin
    ↓
GUI to inspect/manage PostgreSQL
```

## Port mapping

```
"5432:5432"

HOST        CONTAINER
5432    →   5432
```

```
"5433:5432"

HOST        CONTAINER
5433    →   5432
```

## Troubleshooting

```
Can't connect?
      ↓
Is Docker running?
      ↓
Is PostgreSQL container running?
      ↓
docker ps
      ↓
Check docker logs
      ↓
Check port mapping
      ↓
Is local PostgreSQL using 5432?
      ↓
Stop it OR use 5433
      ↓
Check Spring Boot datasource URL
```